

장애 대응 / 복구 드릴 따라하기 가이드

5차시에서 완성한 Olist 레이크하우스 스택(Kafka → Flink → Paimon → Iceberg → Spark → StarRocks/BI)을 직접 망가뜨리고 다시 정상으로 되돌리는 실습입니다. 모든 명령과 예상 출력은 실제 스택에서 검증한 값입니다.

“어디까지 정상이고, 어디부터 의심해야 하는지 증거로 말한다.”

실행 위치 `sessions/05-project-mvp`

브로커 `kafka, kafka2 (2대)`

기준 count `16,693 / 1,971 / 2,000`

mart `olist_category_daily = 759`

라운드 목차

- 0. 기준 상태(Baseline)
- R1 · TaskManager 장애
- R2 · savepoint 재배포
- R3 · Kafka 토픽 내구성 설정 오류·ISR 데모
- R4 · 잘못된 payload
- R5 · Iceberg mart empty
- R5b · snapshot rollback (확장)
- R6 · StarRocks metadata refresh
- R7 · small file 컴팩션 (확장)
- R8 · Spark driver OOM (시연)
- 인시던트 노트 / 제출

공통 진행 리듬: 모든 라운드는 장애 주입 → 증상 관찰 → 원인 가설 → 복구 → 증거 캡처 순서로 진행합니다. 막히면 멈춰서 고치기보다 실패한 명령어와 에러 메시지를 남기세요. 실패 로그도 정상 산출물입니다.

0. 기준 상태(Baseline) 만들기

수업 시작 전(또는 첫 10분)에 아래 상태를 맞춥니다. 모든 명령은 `sessions/05-project-mvp` 에서 실행합니다.

```
# 레포 루트에서 시작
cd sessions/05-project-mvp

git pull origin main
docker compose -f docker-compose.lite.yml up -d --build      # kafka, kafka2 포함 기동
docker compose -f docker-compose.lite.yml ps

./scripts/reset-olist-kafka-topics.sh      # 토픽 RF=2 + min.insync.replicas=2
./scripts/reset-olist-paimon.sh
./scripts/run-flink-olist-paimon-streaming.sh all
./scripts/produce-olist-ux-events.sh
./scripts/produce-olist-review-events.sh
./scripts/produce-olist-order-events.sh
./scripts/start-airflow.sh
./scripts/trigger-airflow-pipeline.sh      # Airflow UI에서 success 확인
./scripts/ops-baseline-evidence.sh
```

정상 기준 (이 값이 baseline)

계층	확인 방법	기준값
Kafka	<code>kafka-topics --describe</code>	broker 2대(<code>de5-kafka</code> , <code>de5-kafka2</code>), olist 토픽 RF=2 / Isr: 1,2
Flink	<code>./scripts/ops-flink-list-jobs.sh</code>	<code>ingest-ux-events</code> , <code>ingest-review-current</code> , <code>ingest-order-current</code> 3개 RUNNING
Paimon	<code>./scripts/query-olist-paimon.sh</code>	16,693 / 1,971 / 2,000
Iceberg	<code>./scripts/query-iceberg-tables.sh</code>	mart 7개 조회 가능, <code>olist_category_daily</code> 759 행
StarRocks/BI	<code>./scripts/query-bi-metrics.sh</code>	external catalog로 조회, <code>olist_category_daily</code> 759
Airflow	<code>localhost:8080</code> (admin/admin)	<code>de5_olist_project_mvp_pipeline</code> 최근 run success

Airflow UI <http://localhost:8080> · Streamlit BI <http://localhost:8501> · Flink UI <http://localhost:8081>

R1

TaskManager 장애

검증됨

의도 · 실제 운영 형태

디스크/리소스 압박으로 kubelet이 TaskManager pod를 evict → slot 상실 → 소비 중단 → backlog/lag 증가. 로컬에서는 TaskManager를 직접 멈춰 **eviction** 이후 효과만 축소 재현하고, checkpoint 기반 복구를 관찰합니다.

주입

```
./scripts/ops-r1-stop-taskmanager.sh
```

관찰

```
docker compose -f docker-compose.lite.yml ps flink-jobmanager flink-taskmanager
./scripts/ops-flink-list-jobs.sh           # 실행 잡이 3 → 0 (slot 없음/restarting)
./scripts/ops-kafka-topic-offsets.sh ux-events
```

관찰 결과: TaskManager 중지 직후 `flink list -r` 의 RUNNING 잡이 0이 됩니다(슬롯을 못 받아 대기).

복구

```
./scripts/ops-r1-start-taskmanager.sh
./scripts/ops-flink-list-jobs.sh      # 잠시 후 3개 RUNNING 자동 복구
./scripts/query-olist-paimon.sh      # count 그대로 (16,693 ...)
```

관찰 결과: TaskManager 재기동 후 ~15초 내 3잡이 **checkpoint**에서 자동 복구되어 RUNNING.

포인트

RUNNING/RESTARTING 상태만으로는 끝이 아닙니다. TaskManager 상태, job 상태, checkpoint/restart 로그, Paimon count를 교차검증합니다. lag/offset은 하나의 신호일 뿐 단독으로 원인을 확정하지 않습니다.

타이밍 주의: restart-strategy는 `fixed-delay` / 3회 입니다. TaskManager를 1~2분 내로 짧게 내렸다 올리세요. 오래 두면 재시작 3회가 소진되어 잡이 FAILED로 남고, 그때는 `run-flink-olist-paimon-streaming.sh all` 재제출(또는 전체 리셋)이 필요합니다.

의도 · 실제 운영 형태

운영 복구는 "다시 실행"이 아니라 "어느 상태에서 이어갈지"를 명시하는 일입니다. clean savepoint 복구는 **전원 핸드온**, 깨진 savepoint 복구는 **멘토 시연**으로 분리합니다.

주입 (UX APPEND JOB 하나만 SAVEPOINT로 정지)

```
./scripts/ops-r2-stop-with-savepoint.sh ingest-ux-events  
# savepoint 경로가 .ops-r2-last-savepoint 에 저장됨 → restore 시 경로 생략 가능
```

관찰 결과: {"status":{"id":"COMPLETED"},"operation":
{"location":"file:/opt/flink/savepoints/savepoint-..."}} 출력 후 잡이 cancel되어 RUNNING 3
→ 2.

복구 (KEEP - SAVEPOINT에서 이어받기)

```
./scripts/ops-r2-restore-job-from-savepoint.sh ingest-ux-events  
./scripts/ops-flink-list-jobs.sh # ingest-ux-events 새 JobID로 RUNNING → 다시
```

멘토 시연 (DISCARD - 깨진 SAVEPOINT)

```
./scripts/ops-r2-restore-bad-savepoint-demo.sh ingest-ux-events  
# 의도적으로 잘못된 savepoint 경로 → 복구 실패 재현
```

포인트

이어받을 state가 깨져 있으면 resume은 복구가 아니라 같은 실패의 반복입니다. 이때는 상태를 버리고(stateless) 다시 시작한 뒤 원천에서 재처리하거나 staging 검증 후 cutover하는 판단이 필요합니다. 학생 핸드온 = KEEP, 멘토 시연 = DISCARD.

의도 · 설정 오입력 사고에서 착안한 재현

실제 사고 형태는 **토픽 설정값 오입력**입니다 — retention 같은 값을 `min.insync.replicas` 에 잘못 넣어 값이 비정상적으로 커지면 `acks=all` producer가 기준을 못 채워 쓰기가 막힙니다. broker 장애·replica 지연으로 ISR이 줄어도 같은 증상(`NotEnoughReplicasException`)이 납니다. 데이터 유실이 아니라 "내구성 기준을 못 맞춰 쓰기를 막는" 안전장치죠. **이 라운드는 실제 사고 그대로가 아니라**, 그 원리(`ISR < min.insync.replicas`)를 로컬에서 보기 쉽게 broker를 내려 만든 **재현 데모**입니다.

전제 확인

```
docker compose -f docker-compose.lite.yml exec -T kafka \
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:19092 --describe --topic
# ReplicationFactor: 2, Isr: 1,2 이면 정상
```

주입 (두 번째 **BROKER** 정지)

```
./scripts/ops-r3-break-kafka-isr.sh          # kafka2 stop → Isr 2→1
./scripts/produce-olist-ux-events.sh --max-events 200
```

관찰 결과 — producer가 비0으로 종료하며 다음을 출력:

```
producer failed: 200 delivery failure(s)
# 원인: NotEnoughReplicasException (Isr 1 < min.insync.replicas 2)
# Kafka offset은 늘지 않음 = 입력 계층(쓰기)에서 막힘. 읽기(Flink)는 계속 정상.
```

복구 (**BROKER** 재기동)

```
./scripts/ops-r3-fix-kafka-isr.sh          # kafka2 start → Isr 1→2 회복
./scripts/produce-olist-ux-events.sh --max-events 200  # 이번엔 성공
```

포인트

producer 실패가 항상 "코드 버그"는 아닙니다. topic 내구성 설정 + broker 가용성 문제일 수 있습니다.

주의: 단일 broker(RF=1)에서는 `min.insync.replicas=2` 를 걸어도 쓰기가 그냥 성공합니다 (ISR이 항상 1뿐이라 기준이 적용 안 됨). 그래서 이 라운드는 `kafka2` 가 반드시 떠 있어야 성립합니다.

R4

잘못된 payload 주입

검증됨

의도 · 실제 운영 형태

schemaless source에서 특정 batch부터 타입/필드가 달라져 parser 또는 sink가 실패하는 상황.
잘못된 `price` 타입 이벤트를 Kafka에 넣고 Flink가 어떻게 반응하는지 봅니다.

주입

```
./scripts/ops-r4-produce-bad-ux-event.sh # price="BROKEN_PRICE" 이벤트 1건
```

관찰 (RESET 전에 RAW PAYLOAD / 로그를 먼저 본다)

```
./scripts/ops-flink-list-jobs.sh # ingest-ux-events 가 사라짐(실패/restart)
docker compose -f docker-compose.lite.yml logs --tail=120 flink-taskmanager
```

관찰 결과 — TaskManager 로그:

```
Caused by: java.lang.NumberFormatException: For input string: 'BROKEN_PRICE'. Over
at ...BinaryStringDataUtil.toDecimal(...)
# UX job은 price를 DECIMAL로 엄격하게 CAST → bad record에서 operator가 실패
```

복구 (CLEAN RESET — KAFKA는 불변이므로 원천을 다시 만든다)

```
# 실행 중인 잡 취소 후 토픽/Paimon 리셋 → 재적재 → DAG
./scripts/reset-olist-kafka-topics.sh
./scripts/reset-olist-paimon.sh
./scripts/run-flink-olist-paimon-streaming.sh all
./scripts/produce-olist-ux-events.sh && ./scripts/produce-olist-review-events.sh &
./scripts/trigger-airflow-pipeline.sh
./scripts/query-olist-paimon.sh # 16,693 / 1,971 / 2,000 복구
```

포인트

Kafka에 이미 들어간 잘못된 메시지는 수정되지 않습니다. 이 라운드의 증거는 **Kafka raw payload**와 **Flink log**입니다. 운영형 fix는 전체 리셋이 아니라 **bad record quarantine** 또는 **tolerant parsing**입니다(위 리셋은 수업용 빠른 회복 편의 조치).

참고: R4는 복구가 전체 리셋이라 가장 무겁습니다(~3분). 진행 방식은 멘토 안내에 따르세요(직접 핸드온 또는 시연 관찰).

R5

Iceberg mart empty / 누락

검증됨

의도 · 실제 운영 형태

DAG는 성공처럼 보였지만 BI가 읽는 Iceberg mart가 비어 BI에서 장애가 드러난 사건. **"DAG success가 곧 BI 정상은 아니다"**가 핵심 메시지입니다.

주입 (DROP이 아니라 ROW만 비움)

```
./scripts/ops-r5-empty-iceberg-mart.sh
./scripts/query-iceberg-tables.sh      # olist_category_daily COUNT(*) = 0
./scripts/query-bi-metrics.sh          # StarRocks 기준도 category_daily = 0
```

관찰 결과 — Spark/Iceberg와 StarRocks 모두 **759 → 0**. 마지막 Airflow DAG run은 여전히 **success**로 남아 있습니다(주입은 DAG 이후에 일어나고, validate 단계는 이 테이블을 검사하지 않음).

BI 화면 주의: Streamlit BI를 열면 상단 KPI/차트는 정상이지만 맨 아래 **"Analytics Table"** 패널이 **KeyError** 예외로 깨집니다(빈 데이터프레임 컬럼 인덱싱). "0건으로 깔끔히 빈 화면"이 아니라 예외 크래시로 보입니다. 깔끔한 증거가 필요하다면 화면 대신 `query-iceberg-tables.sh` / `query-bi-metrics.sh` 의 0 출력을 쓰세요.

복구 기준점 확인 (ROLLBACK 실행 전 단계)

```
./scripts/ops-r5-find-iceberg-recovery-point.sh
# 출력: RECOVERY_POINT_CANDIDATE snapshot_id=<id> row_count=759
```

복구 - 방법 A: REBUILD (권위 있는 정답, 원천에서 재DERIVE)

```
./scripts/trigger-airflow-pipeline.sh      # DAG가 mart 재생성 → 759
./scripts/query-iceberg-tables.sh
./scripts/query-bi-metrics.sh
```

포인트

Flink/Paimon이 정상이고 DAG가 초록이어도 mart가 비면 BI는 실패합니다. 현재 프로젝트는 `validate_bi_metric_counts` 로 일부 지표를 검증하도록 보강했지만, 모든 mart를 검사하지는 않습니다. drop이 아니라 empty인 이유는 실제 사례가 "테이블 없음"이 아니라 "테이블은 있는데 데이터가 없음"이기 때문입니다.

R5b

snapshot rollback (확장)

확장 · 검증됨

의도

R5의 두 번째 복구 방법. mart가 깨졌을 때 원천에서 다시 만드는 대신, Iceberg snapshot 포인터를 직전 정상 snapshot으로 즉시 되돌립니다.

복구 - 방법 B: ROLLBACK (즉시)

```
# R5에서 ops-r5-empty로 비운 뒤, find-recovery-point가 준 snapshot_id 사용
./scripts/ops-r5b-rollback-iceberg-mart.sh <snapshot_id>
./scripts/query-iceberg-tables.sh                # 0 → 759 즉시 복구
./scripts/query-bi-metrics.sh                     # StarRocks도 759
```

포인트 - REBUILD VS ROLLBACK

rebuild(R5·Airflow): 원천(Paimon)에서 다시 derive하는 권위 있는 복구. 느리고 원천이 정상이어야 함.

rollback(R5b): metadata 포인터만 과거 snapshot으로. 즉시지만 그 snapshot 이후 commit을 전부 폐기함 → "그래도 되는 상황"에서만.

find-recovery-point는 "어느 snapshot까지 정상인가"를 확정하는 단계이고, rollback은 그 다음의 결정·실행입니다.

R6

StarRocks external metadata refresh

검증됨

의도 · 실제 운영 형태

native table은 정상인데 조회 계층(StarRocks/Iceberg-compatible view)이 최신 상태를 못 보는 문제. 독립 파괴 주입이 아니라 R5/rebuild 이후 serving 계층을 다시 맞추는 **검증 라운드**입니다.

관찰 / 복구

```
./scripts/query-bi-metrics.sh          # refresh 전 숫자
./scripts/ops-r6-refresh-starrocks-iceberg.sh
./scripts/query-bi-metrics.sh          # refresh 후 숫자
```

포인트

이 스택의 StarRocks Iceberg external catalog는 거의 항상 fresh합니다(조회 시 catalog를 재생성). 따라서 **refresh 전후 숫자가 같으면 정상**이며, "이미 fresh했다"는 결론이지 실패가 아닙니다. 그래도 "조회 계층은 원천과 별도로 확인해야 한다"는 교훈은 동일합니다. 가능하면 Paimon/Iceberg native count, StarRocks count, BI metric을 3자 비교하세요.

R7

small file 문제와 컴팩션 (확장)

확장 · 검증됨

의도 · 실제 운영 형태

스트리밍/빈번 commit은 작은 데이터 파일을 양산합니다. 파일이 수천 개가 되면 read planning/메타데이터 비용이 커져 쿼리가 느려집니다. 정기 컴팩션이 운영 유지보수 항목입니다. baseline 마트는 건드리지 않고 별도 iceberg_lake.opsdemo 테이블만 씁니다.

주입 (작은 COMMIT 다수 → SMALL FILE)

```
./scripts/ops-r7-make-small-files.sh
```

관찰 결과: 행 20개에 데이터 파일 **20개**(INSERT 1건 = commit 1개 = 작은 파일 1개) = read amplification.

복구 (컴팩션)

```
./scripts/ops-r7-compact-small-files.sh      # rewrite_data_files → 파일 20 → 1  
./scripts/ops-r7-reset-small-files.sh        # 데모 테이블 정리
```

관찰 결과: 파일 수 **20 → 1**, 행 수는 그대로(20). 비용만 줄고 정답은 안 바뀜.

포인트

작은 파일이 많다는 것은 데이터가 틀렸다는 뜻이 아니라 읽기 비용이 새고 있다는 뜻입니다. 컴팩션 (rewrite_data_files / Paimon compaction)은 정답을 바꾸지 않고 비용을 줄이는 정기 유지보수입니다.

R8

Spark driver OOM

멘토 시연

의도 · 실제 운영 형태

Spark driver/executor가 collect 과다·스큐·큰 셔플로 OOM 나는 상황. 데이터 손상이 아니라 컴퓨트 자원 장애이며, 진단은 로그/메트릭, 복구는 쿼리·자원 설정 수정입니다. **멘토 시연 전용**(학생 핸즈온 아님).

시연

```
./scripts/ops-r8-spark-driver-oom-demo.sh    # --driver-memory 512m + 큰 collect()
```

관찰 결과:

```
: java.lang.OutOfMemoryError: Java heap space
# OOM은 이 spark-submit JVM(-Xmx) 안에 갇혀 다른 컨테이너에는 영향 없음
```

포인트

OOM 복구의 시작은 "메모리를 늘리자"가 아니라 "왜 그렇게 많이 필요했나"입니다. `collect()` 회피 / 집계·limit / 스큐를 먼저 의심합니다.

안전 주의: 실행 환경은 컨테이너 메모리 제한이 없고 호스트 메모리를 공유합니다. `--driver-memory` 로 캡을 건 Spark driver OOM만 안전하며, **캡 없는 OOM(특히 Flink TaskManager OOM)**은 전체 스택을 위협하므로 주입하지 않습니다.

인시던트 노트 / 제출

각 라운드 인시던트 노트 (4줄 + R5는 1줄 추가)

장애:
관찰한 증상:
확인한 증거:
복구 명령/결과:
R5에서는 한 줄 추가
복구 기준점 후보 snapshot_id:

이번 주부터 과제는 "제출하고 끝"이 아니라 최종 발표 자료로 쌓아갑니다. 같은 공통 프로젝트라도 어떤 증거를 중요하게 봤고 어떤 운영 리스크를 발견했는지가 개인 포트폴리오의 차별점입니다.

막혔을 때

수업 중 막히면 멈춰서 혼자 고치려 하기보다, **실패한 명령어와 에러 메시지를 남기고 멘토를 부르세요**. 실패 로그도 이번 차시의 정상 산출물입니다. 지금 어느 계층까지 정상인지부터 빠르게 확인하면 원인을 좁히기 쉽습니다.

```
./scripts/ops-baseline-evidence.sh # Kafka/Flink/Paimon/Iceberg/StarRocks 상태를 한 번에 확인
```

복구가 꼬였다고 느껴지면 직접 여러 reset를 섞지 말고, 위 증거부터 캡처한 뒤 멘토와 함께 어느 계층만 되돌릴지 정하세요.

마지막 정리

```
./scripts/ops-baseline-evidence.sh # 모든 계층 baseline 재확인
```

DE5 Commerce Lakehouse · 6차시 운영 장애 대응 / 복구 드릴 · 모든 명령과 출력은 실제 스택에서 검증한 값입니다.

실행 위치: sessions/05-project-mvp · 상세 텍스트 자료: README.md · COMMAND_CARD.md · REAL_INCIDENT_MAP.md